

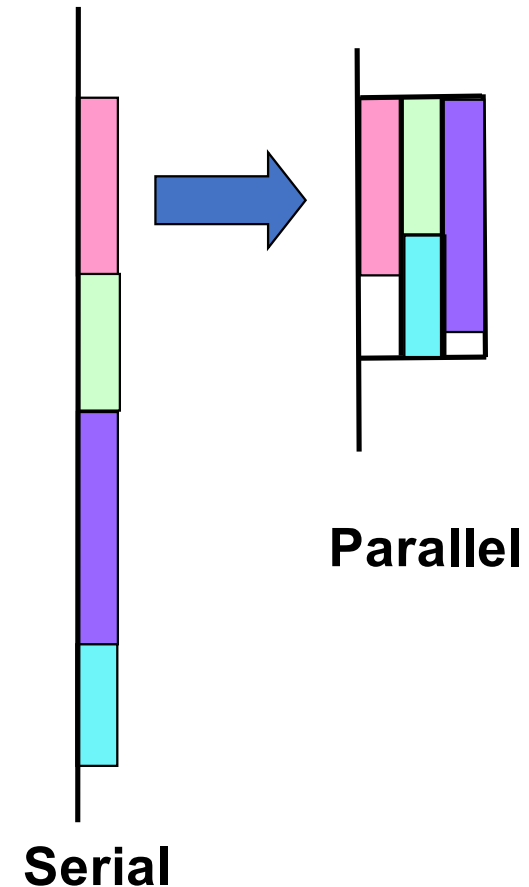
Advanced OpenMP Tasks



What are OpenMP tasks?

- Tasks are independent units of work
- Tasks are composed of:
 - a block of code to execute
 - data to compute with (per-task private copies)
- Tasks are assigned to threads for execution.
 - programmer has little control over how this is done

|epcc|



OpenMP tasks

- The task construct includes a structured block of code
- Inside a parallel region, a thread encountering a task construct will package up the code block and its data for execution
- Some thread in the parallel region will execute the task at some point in the future
 - note: could be encountering thread, right now
- Tasks can be nested: i.e. a task may itself generate tasks.

task directive

Syntax:

Fortran:

```
!$OMP TASK [clauses]
```

structured block

```
!$OMP END TASK
```

C/C++:

```
#pragma omp task [clauses]
```

structured-block

Example

```
#pragma omp parallel  
{  
    #pragma omp master  
    {  
        #pragma omp task  
        fred();  
        #pragma omp task  
        daisy();  
        #pragma omp task  
        billy();  
        #pragma omp task  
        molly();  
    }  
}
```

Create some threads

Thread 0 packages tasks

Tasks executed by some thread in some order

Example (Fortran)

```
!$omp parallel  
  !$omp master  
    !$omp task  
      call fred()  
    !$omp end task  
    !$omp task  
      call daisy()  
    !$omp end task  
    !$omp task  
      call billy()  
    !$omp end task  
    !$omp task  
      call molly()  
    !$omp end task  
  !$omp end master  
!$omp end parallel
```

Create some threads

Thread 0 packages tasks

Tasks executed by some thread in some order

When/where are tasks complete?

- At thread barriers (explicit or implicit)
 - applies to all tasks generated in the current parallel region up to the barrier
- At taskwait directive
 - i.e. Wait until all tasks defined in the current task have completed.
 - Fortran: `!$OMP TASKWAIT`
 - C/C++: `#pragma omp taskwait`
 - Note: applies only to tasks generated in the current task, not to “descendants” .
 - The code executed by a thread in a parallel region is considered a task here

When/where are tasks complete?

- At the end of a taskgroup region

- Fortran:

```
! $OMP TASKGROUP
```

structured block

```
! $OMP END TASKGROUP
```

- C/C++:

```
#pragma omp taskgroup
```

structured-block

- wait until all tasks created within the taskgroup have completed
 - applies to all “descendants”

Example



```
#pragma omp parallel
{
    #pragma omp master
    {
        #pragma omp task
        fred();
        #pragma omp task
        daisy();
        #pragma omp taskwait
        #pragma omp task
        billy();
    }
}
```

fred() and
daisy() must
complete before
billy() starts

A blue arrow points from the text box to the `#pragma omp taskwait` line in the code block.

Example (Fortran)



```
!$omp parallel
  !$omp master
    !$omp task
      call fred()
    !$omp end task
    !$omp task
      call daisy()
    !$omp end task
    !$omp taskwait
    !$omp task
      call billy()
    !$omp end task
  !$omp end master
!$omp end parallel
```

`fred()` and
`daisy()` must
complete before
`billy()` starts

Linked list traversal

```
p = listhead ;  
while (p) {  
    process(p) ;  
    p=next(p) ;  
}
```

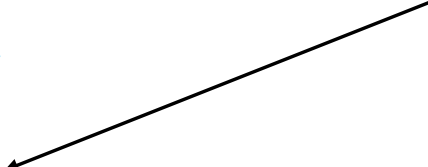
```
p = listhead  
do while (p /= 0)  
    call process(p)  
    p=next(p)  
end do
```

- Classic linked list traversal
- Do some work on each item in the list
- Assume that items can be processed independently
- Cannot use an OpenMP loop directive

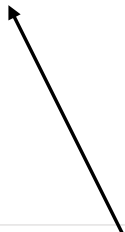
Parallel linked list traversal

```
#pragma omp parallel
{
    #pragma omp master
    {
        p = listhead ;
        while (p) {
            #pragma omp task firstprivate(p)
            {
                process (p) ;
            }
            p=next(p) ;
        }
    }
}
```

Only one thread
packages tasks



makes a copy of **p**
when the task is
packaged



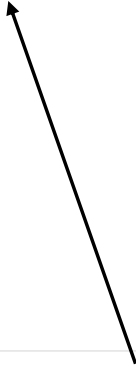
Parallel linked list traversal

```
!$omp parallel
  !$omp master
    p = listhead
    do while (p /= 0)
      !$omp task firstprivate(p)
        call process (p)
      !$omp end task
      p=next(p)
    end do
  !$omp end master
!$omp end parallel
```

Only one thread
packages tasks



makes a copy of **p**
when the task is
packaged



Parallel linked list traversal

Thread 0:

```
p = listhead ;  
while (p) {  
  < package up task >  
  p=next (p) ;  
}  
  
while (tasks_to_do){  
  < execute task >  
}  
  
< barrier >
```

Other threads:

```
while (tasks_to_do) {  
  < execute task >  
}  
  
< barrier >
```

Parallel pointer chasing on multiple lists

```
#pragma omp parallel
{
    #pragma omp for private(p)
    for ( int i =0; i <numlists; i++) {
        p = listheads[i] ;
        while (p ) {
            #pragma omp task firstprivate(p)
            {
                process (p) ;
            }
            p=next (p) ;
        }
    }
}
```

All threads package
tasks



Data scoping with tasks

- Variables can be shared, private or firstprivate with respect to a task
- These concepts are a little bit different compared with threads:
 - If a variable is **shared** on a task construct, the references to it inside the construct are to the storage with that name at the point where the task was encountered
 - If a variable is **private** on a task construct, the references to it inside the construct are to new uninitialized storage that is created when the task is executed
 - If a variable is **firstprivate** on a construct, the references to it inside the construct are to new storage that is created and initialized with the value of the existing storage of that name when the task is encountered

Data scoping defaults

- The behavior you want for tasks is usually firstprivate, because the task may not be executed until later (and variables may have gone out of scope)
 - Variables that are private when the task construct is encountered are firstprivate by default
- Variables that are shared in all constructs starting from the innermost enclosing parallel construct are shared by default

```
#pragma omp parallel shared(A) private(B) !$omp parallel shared(A, C) private(B)
{
    ...
    #pragma omp task
    {
        int C;
        compute(A, B, C);
    }
}
```

```
...
!$omp task private(C)
    call compute(A, B, C)
!$omp end task
!$omp end parallel
```

Inside the task, A is shared, B is firstprivate, C is private

Example: Fibonacci numbers

$F_n = F_{n-1} + F_{n-2}$ (inefficient $O(n^2)$ recursive implementation!)

```
int fib ( int n )
{
  int x,y;
    if ( n < 2 ) return n;
  x = fib(n-1);
  y = fib(n-2);
  return x+y;
}
```

```
int main()
{
  int NN = 5000;
  int res = fib(NN);
}
```

```
RECURSIVE FUNCTION fib(n) RESULT(fibo)
  INTEGER, INTENT(IN) :: n
  INTEGER :: fibo, x, y
  IF (n < 2) THEN
    fibo = n
  ELSE
    x = fib(n-1)
    y = fib(n-2)
    fibo = x+y
  END IF
END FUNCTION fib
```

```
PROGRAM FIBMAIN
  INTEGER :: NN, res
  NN = 5000
  res = fib(NN)
END PROGRAM FIBMAIN
```

Parallel Fibonacci

```
int fib ( int n )
{
    int x,y;
    if ( n < 2 ) return n;
    #pragma omp task shared(x)
    x = fib(n-1);
    #pragma omp task shared(y)
    y = fib(n-2);
    #pragma omp taskwait
    return x+y;
}

int main()
{
    int NN = 5000;
    #pragma omp parallel
    {
        #pragma omp master
        int res = fib(NN);
    }
}
```

- Binary tree of tasks
- Traversed using a recursive function
- A task cannot complete until all tasks below it in the tree are complete (enforced with taskwait)
- **x,y** are local, and so private to current task
 - must be shared on child tasks so they don't create their own firstprivate copies at this level!

Parallel Fibonacci



```
RECURSIVE FUNCTION fib(n) RESULT(fibo)
  INTEGER, INTENT(IN) :: n
  INTEGER :: fibo, x, y
  IF (n < 2) THEN
    fibo = n
  ELSE
    !$omp task shared(x)
      x = fib(n-1)
    !$omp end task
    !$omp task shared(y)
      y = fib(n-2)
    !$omp end task
    !$omp taskwait
      fibo = x+y
  END IF
END FUNCTION fib
```

```
PROGRAM FIBMAIN
  INTEGER :: NN, res
  NN = 5000
  !$omp parallel
  !$omp master
    res = fib(NN)
  !$omp end master
  !$omp end parallel
END PROGRAM FIBMAIN
```

Using tasks

- Getting the data attribute scoping right can be quite tricky
 - default scoping rules different from other constructs
 - as ever, using `default(none)` is a good idea
- Don't use tasks for things already well supported by OpenMP
 - e.g. standard do/for loops
 - the overhead of using tasks is greater
- Don't expect miracles from the runtime
 - best results usually obtained where the user controls the number and granularity of tasks

Parallel pointer chasing again

```
#pragma omp parallel
{
    #pragma omp master private(p)
    {
        p = listhead ;
        while (p) {
            #pragma omp task firstprivate(p)
            {
                process (p,nitems);
            }
            for (i=0; i<nitems &&p; i++){
                p=next(p);
            }
        }
    }
}
```

process
nitems at
a time

skip **nitems** ahead
in the list

Parallel Fibonacci again

```
int fib ( int n )
{
    int x,y;
    if ( n < 2 ) return n;
    #pragma omp task shared(x) if(n > LEVEL)
        x = fib(n-1);
    #pragma omp task shared(y) if(n > LEVEL)
        y = fib(n-2);
    #pragma omp taskwait
        return x+y;
}

int main()
{
    int NN = 5000;
    #pragma omp parallel
    {
        #pragma omp master
            int res = fib(NN);
    }
}
```

- Stop creating tasks at some level in the tree.

Taskloop

taskloop directive provides a convenient way to turn loop iterations into tasks

```
#pragma omp taskloop grainsize(16)
for (int i=0; i<n; i++){
    a[i] = somefunc(i);
}
```

```
!$omp taskloop grainsize(16)
do i = 1,N
    a(i) = somefunc(i)
end do
```

- Each set of 16 consecutive iterations will form a task
- Can specify the number of tasks with the **num_tasks** clause instead of grainsize

Task dependencies

- **depend** clause on task construct

```
#pragma omp task depend(type:list)
```

```
!$omp task depend(type:list)
```

where *type* is **in**, **out** or **inout** and *list* is a list of variables.

- list may contain subarrays: OpenMP 4.0 includes a syntax for C/C++

- **in**: the generated task will be a dependent task of all previously generated sibling tasks that reference at least one of the list items in an **out** or **inout** clause.
- **out** or **inout**: the generated task will be a dependent task of all previously generated sibling tasks that reference at least one of the list items in **in**, **out** or **inout** clause.

Example

```
#pragma omp task depend (out:a)
{ ... }
#pragma omp task depend (out:b)
{ ... }
#pragma omp task depend (in:a,b)
{ ... }
```

- The first two tasks can execute in parallel
- The third task cannot start until both the first two are complete

Task reductions

- Cannot simply use reduction variables inside tasks:

```
#pragma omp parallel reduction(+:a)
{
  #pragma omp single
  {
    for (i=0;i<n;i++) {
      #pragma omp task
      {
        a++;
      }
    }
  }
}
```

Not allowed!

```
#pragma omp parallel shared(a)
{
  #pragma omp single
  {
    for (i=0;i<n;i++) {
      #pragma omp task
      {
        #pragma omp atomic
        a++;
      }
    }
  }
}
```

Allowed but possibly inefficient

Task reductions

- Can use a taskgroup like this:

```
#pragma omp parallel shared(a)
{
#pragma omp single
{
#pragma omp taskgroup task_reduction(+:a)
{
    for (i=0;i<n;i++) {
#pragma omp task in_reduction(+:a)
        {
            a++;
        }
    }
} // reduction happens at end of taskgroup
}
}
```

Reusing this material



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

This means you are free to copy and redistribute the material and adapt and build on the material under the following terms: You must give appropriate credit, provide a link to the license and indicate if changes were made. If you adapt or build on the material you must distribute your work under the same license as the original.

Note that this presentation contains images owned by others. Please seek their permission before reusing these images.

